

Monolithic Application Analysis

Challenge goal: analyze the code and identify why it is considered a "monolith", and discuss the advantages and disadvantages of this approach for an MVP.

Application overview

The ToggleMaster MVP is a REST API for managing Feature Flags, composed of:

- `app.py` : a single Flask application containing all responsibilities: API routes, database access, business rules, and initialization.
- PostgreSQL**: relational database with a single table (`flags`).
- `Dockerfile` / `docker-compose.yaml` : packaging of the application and database for local execution.

Endpoints

Method	Endpoint	Description
GET	<code>/health</code>	Application health check
POST	<code>/flags</code>	Creates a feature flag
GET	<code>/flags</code>	Lists all flags
GET	<code>/flags/<name></code>	Returns the status of a flag
PUT	<code>/flags/<name></code>	Updates the status of a flag

Why is it a monolith?

- A single process holds all layers.** In `app.py` , HTTP handling, business rules, and database access are mixed together within the same functions. There is no separation into modules, layers, or repositories.
- A single deployment unit.** Everything scales, ships, and fails together: `gunicorn app:app` starts the entire platform. There is no way to deploy or scale only one part of the application

separately. Running `docker compose restart` on the `app` container restarts the application as a whole and, while the process is down, the platform is completely unavailable because there are no replicas.

Advantages of the monolithic approach for an MVP

1. **Ease of understanding:** a developer can understand how the system works in a short time. No network between services, no serialization, no eventual consistency, no distributed tracing. A single stack trace is enough to see the complete picture.
2. **Validation speed:** the deploy is `docker compose up`. The goal of an MVP is to validate the idea, not to scale, and the monolith delivers exactly that.
3. **Trivial operation:** one service to monitor, one log, one healthcheck. Everything is centralized.
4. **Minimum cost:** in the case of the Tech Challenge, we need nothing more than a small container to host the application.

Disadvantages and limitations of the monolithic approach

1. **All-or-nothing scaling:** if one function becomes overloaded and needs scaling, we have to scale the entire platform, including the parts that do not need it.
2. **Single release cycle:** changing a validation forces the redeployment of everything. On large teams, this leads to merge conflicts in the same file and collateral regressions.
3. **SPOF:** single point of failure. If the process goes down, the platform goes down completely.

Conclusion

Within the scope covered by PHASE 1, the Monolith is the right decision. The MVP needs fast feedback, and premature decomposition into microservices would kill the timing (Martin Fowler's "MonolithFirst" idea). The real problem is not the monolith, but letting it rot: the healthy evolution path would be first a modular monolith (separating `routes/`, `services/`, and `repository/` within the same deployment, and adding a connection pool) and only, in the later phases of the project, extracting services when there is scale or enough teams to justify it.

Notes from running locally

Environment: Linux VPS (Rocky Linux 9), Docker 29.8.0 + Docker Compose v5.5.1, cloned from this repository to `/opt/togglemaster` and started with `docker compose up -d --build`. A [Caddy](#) reverse proxy (in `/opt/proxy`) terminates TLS on ports 80/443 with a Let's Encrypt certificate and forwards to the app on port 5000.

Public URL: <https://toggle-local.cheb.com.br>

Endpoint tests (2026-09-03, all passed):

Test	Request	Result
Health check	<code>GET /health</code>	<code>{"status":"ok"}</code> (200)
Create flag	<code>POST /flags {"name":"new-feature","is_enabled":true}</code>	201, flag created
List flags	<code>GET /flags</code>	<code>[{"is_enabled":true,"name":"new-feature"}]</code>
Get flag	<code>GET /flags/new-feature</code>	<code>{"is_enabled":true,"name":"new-feature"}</code>
Update flag	<code>PUT /flags/new-feature {"is_enabled":false}</code>	200, flag updated
Duplicate flag	<code>POST /flags</code> (existing name)	409 with error message
Persistence	<code>docker compose restart</code>	Data preserved (PostgreSQL volume)

Confirmation of monolithic behavior observed during execution: the entire application (routes, business rules, database access) runs as a single process inside one container; the only separate component is the database. Restarting the app container makes the entire platform unavailable until the process is back, and there is no way to deploy or scale parts of it independently.

References

- [Application README](#)
- Challenge statement: `POSTECH - DCLT - Tech Challenge - Fase 1.pdf`